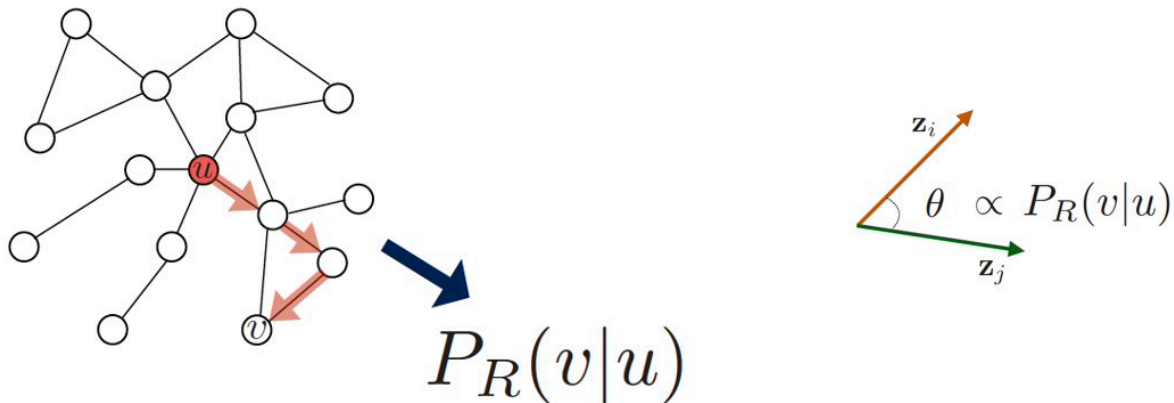


L14: Random-walk Encodings and Node2vec

The previous methods to compute node embeddings are deterministic in nature. More recent embedding methods rely instead on stochastic notions of "neighborhood overlap" between pairs of nodes.

In particular, **DeepWalk** (a misnomer -- because it does not use deep learning) and **node2vec** are the most popular in this category. Both of them construct node embeddings based on *random walks*: *two nodes u and v have similar embeddings ($\mathbf{z}_u \approx \mathbf{z}_v$) if the two nodes tend to co-occur on short random walks over the graph.*



Images from Jure Leskovec, "Representation Learning on Networks" tutorial, WWW 2018

Specifically, assume that we have an undirected network, and suppose that $P_R(v|u)$ is the probability of visiting node v on random walks of length- T starting at node u , where T is typically a small positive integer (say between 2 to 10, depending on the size of the network). If that probability is high, the two nodes are "close" and they should have similar embeddings. Otherwise, their embeddings should be quite different.

Based on this insight, we can parameterize the probability $P_R(v|\mathbf{z}_u)$ using the following "softmax" ratio (which is always between 0 and 1) of the node embeddings:

$$\frac{e^{\mathbf{z}_u^T \mathbf{z}_v}}{\sum_{n \in V} e^{\mathbf{z}_u^T \mathbf{z}_n}} \approx P_R(v|\mathbf{z}_u)$$

We can now compute the node embeddings by minimizing the following cross-entropy loss function

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log P(v|\mathbf{z}_u)$$

where $N_R(u)$ is the multi-set of nodes in the ensemble of random walks of length T that start from u (*it is a multi-set because we can have repeated elements -- nodes can be visited multiple times on each random walk*). So, a node v that appears multiple times in $N_R(u)$ will contribute more in the previous sum than a node v' that appears only once in $N_R(u)$.

If you are not familiar with cross-entropy loss functions, the goal here is to adjust the model parameters (i.e., the node embeddings) so that the terms $P_R(v|\mathbf{z}_u)$ are close to 1 when v is in $N_R(u)$ and close to 0 when v is *not* in $N_R(u)$.

Algorithmically, we can think about the previous process as follows:

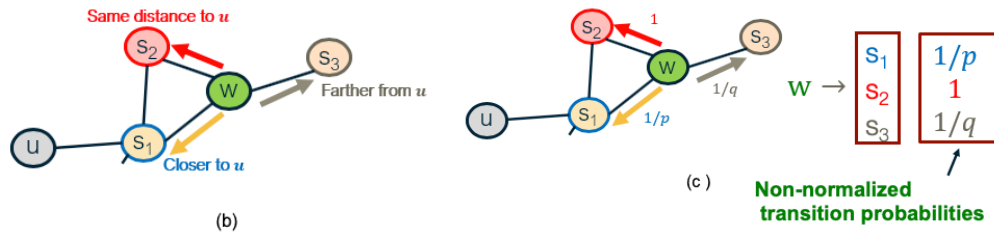
1. We run short random walks of length T starting from each node u on the graph using a given random walk strategy $R(T)$.
2. For each node u , we compute $N_R(u)$, defined as the multi-set of nodes visited on random walks starting from u .
3. We optimize the embeddings $\mathbf{z}_v$ for every node v based on the previous loss function, given that we know $N_R(u)$ for every node u .

Different methods in this literature use various approximations to minimize the previous loss functions, resulting in different computational complexity. For instance, **node2vec** also uses a set of "negative samples" to approximate the denominator of the softmax term so that the corresponding summation does not need to consider all pairs of nodes.

Another difference between *node2vec* and earlier methods is that it introduces two parameters, p and q , that bias the random walk process. Specifically, p controls the probability that the walk will return to the previous node, while q controls the probability that the walk will visit a node's neighbor that it has not visited before. The following example illustrates these random walk strategies.

Suppose that a random walk started at u and is now at w . The neighbors of w can be: closer to u (s_1), farther from u (s_3), same distance to u (s_2). We remember where that walk came from. Where to go next? The two parameters that control that decision are:

- p ... return parameter (**BFS-like** walk: Low value of p)
- q ... "walk away" parameter (**DFS-like** walk: Low value of q)



Images from Jure Leskovec, "Representation Learning on Networks" tutorial, WWW 2018.

Random walk approaches are more *expressive* than the deterministic embedding methods we discussed in the previous page because the notion of "similarity" between node pairs is stochastic and it can incorporate information about both local and more distant neighbors. They are also more *efficient* because the training process only needs to consider node pairs that co-occur on short random walks -- instead of all possible node pairs.

Food For Thought

Suppose that we do not use the previous softmax ratio to parameterize the probabilities $P_R(v|z_u)$. Can you think of another way to parameterize these probabilities as a function of the node embedding vectors?